Day 127: Denoising Autoencoders: a Real Noise-Removal Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 2 | Complete study guide

Watch: 86 - Applications of Autoencoders - Denoising using MNIST dataset (DigitalSreeni) | Lec18 Sparse and Denoising Autoencoders (Deep Learning For Visual Computing - IITKGP)

Script: `day127_denoising_autoencoders.py` · every number in this guide is copied by program from `day127_run_output.txt`, the real output of that script.

A denoising autoencoder sees a damaged image and is trained to output the clean one. On MNIST digits with heavy Gaussian noise it restores them to 19.25 dB, against 17.55 for a median filter, 16.42 for PCA and 12.93 for Day 126's plain autoencoder, which makes the noise worse. The limits are measured too: on light noise the damaged image itself beats every denoiser (22.65 dB), and a denoiser trained on one kind of damage can do harm on another (9.45 dB). Its codes, though, stay useful when the input is damaged: kNN 0.9090 against the plain autoencoder's 0.2815.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

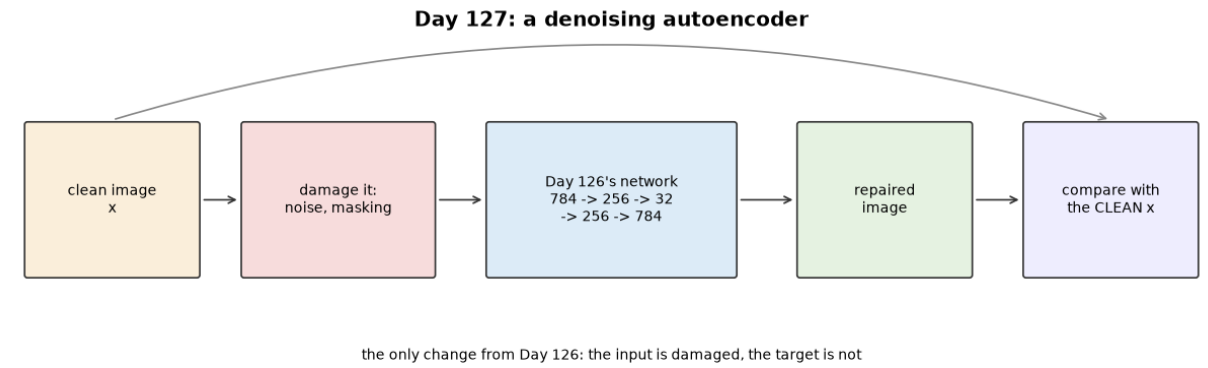
A **denoising autoencoder** is Day 126's network with one change: the input is a **damaged** copy of the image, and the target is the **clean** original. It can no longer just copy; it has to know what digits look like to undo the damage.

Repairs are scored by **PSNR** (peak signal-to-noise ratio): $10 \log_{10}(1 / \text{mean squared error})$ for pixels in [0, 1], in decibels. Higher is better; every 10 dB is ten times less squared error.

REAL-WORLD SCENARIO

Restoring a scanned letter covered in specks: someone who knows what handwriting looks like can tell a speck from a dot on an 'i'. A denoiser learns that knowledge from clean examples it damaged itself.

SYSTEM DIAGRAM



Damage the input, keep the clean target: the only change from Day 126.

Today's part for the week's build

`day127_exercise.py` is the three kinds of damage, PSNR and the median filter, checked by hand. Its real output:

ACTUAL OUTPUT

```
PSNR of an image off by 0.1 everywhere: 20.00 dB; by hand 10 log10(1 / 0.01) = 20.00 dB
masking 0.3 keeps 0.7000 of the pixels (by hand about 0.7)
Gaussian noise on black pixels, clipped: none below 0: True; about half stay 0: 0.5001
hidden squares: [100, 100, 100, 100, 100] pixels each (by hand 10 x 10 = 100), all set to 0: True
median 3 x 3: the lone bright pixel becomes 0; the middle of the 5 x 5 block stays 1
```

Data: MNIST (cached by Day 126), damaged on purpose so the clean answer is known exactly; 20,000 training and 10,000 test images. Baselines: a 3 x 3 median filter and PCA with 32 components (Day 123).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: PSNR, and three kinds of damage

DEFINITION — start here, no prior knowledge assumed

Gaussian noise adds a random amount to every pixel; masking sets half the pixels to 0; a hidden square blanks a 10 x 10 patch.

REAL-WORLD SCENARIO

A photo with film grain, a photo with half its dots missing, and a photo with a sticker on it.

MATHEMATICS

$$PSNR = 10 \log_{10}(1 / MSE),\ MSE = \text{mean of } (\text{repaired} - \text{clean})^2 \text{ over pixels in } [0, 1]$$

IMPLEMENTATION

```
def gaussian(X, sigma, rng):
    return np.clip(X + rng.normal(0, sigma, X.shape).astype(np.float32), 0, 1)
def masking(X, p, rng):
    return X * (rng.uniform(size=X.shape) >= p).astype(np.float32)
def hide_square(X, size=10, rng=None):
    rng = rng or np.random.RandomState(0); Y = X.reshape(-1, 28, 28).copy(); M = np.zeros_like(Y,
        bool)
    for i, (r, c) in enumerate(rng.randint(4, 24 - size, size=(len(Y), 2))):
        Y[i, r:r + size, c:c + size] = 0; M[i, r:r + size, c:c + size] = True
    return Y.reshape(len(X), -1), M.reshape(len(X), -1)
def psnr(R, X):
    return float(10 * np.log10(1.0 / np.mean((R - X) ** 2)))
```

ACTUAL OUTPUT

```
one test image with Gaussian noise (sigma 0.3): mean squared error 0.04445; by hand 10 log10(1 /
    0.04445) = 13.52 dB; psnr() 13.52 dB
(sigma^2 = 0.09 would be the error without clipping; clipping to [0, 1] removes part of it,
    because most pixels are 0)
  Gaussian, sigma 0.3       : PSNR of the damaged test images 13.32 dB
  masking, half the pixels  : PSNR of the damaged test images 12.45 dB
  a hidden 10 x 10 square   : PSNR of the damaged test images 14.15 dB
Actual output from this lesson's run.
```

What it means

One image with $\sigma = 0.3$ noise has squared error 0.04445, so PSNR = $10 \log_{10}(1/0.04445) = 13.52$ dB. Without clipping the error would be about $\sigma^2 = 0.09$; clipping to [0, 1] removes part of it, because most MNIST pixels are 0 and negative noise is cut off. The three kinds of damage leave the test images at 13.32, 12.45 and 14.15 dB.

WHY CHOOSE THIS?

One number for any repair, comparable across methods.

WHY NOT THIS? (TRADEOFFS)

PSNR counts squared error, not whether a digit still looks like itself.

2. Plain against denoising, against the baselines

DEFINITION — start here, no prior knowledge assumed

Three networks trained for 20 epochs each: plain (Day 126), denoising on Gaussian $\sigma = 0.3$, denoising on masking with $p = 0.5$; all scored on the same damaged test images.

REAL-WORLD SCENARIO

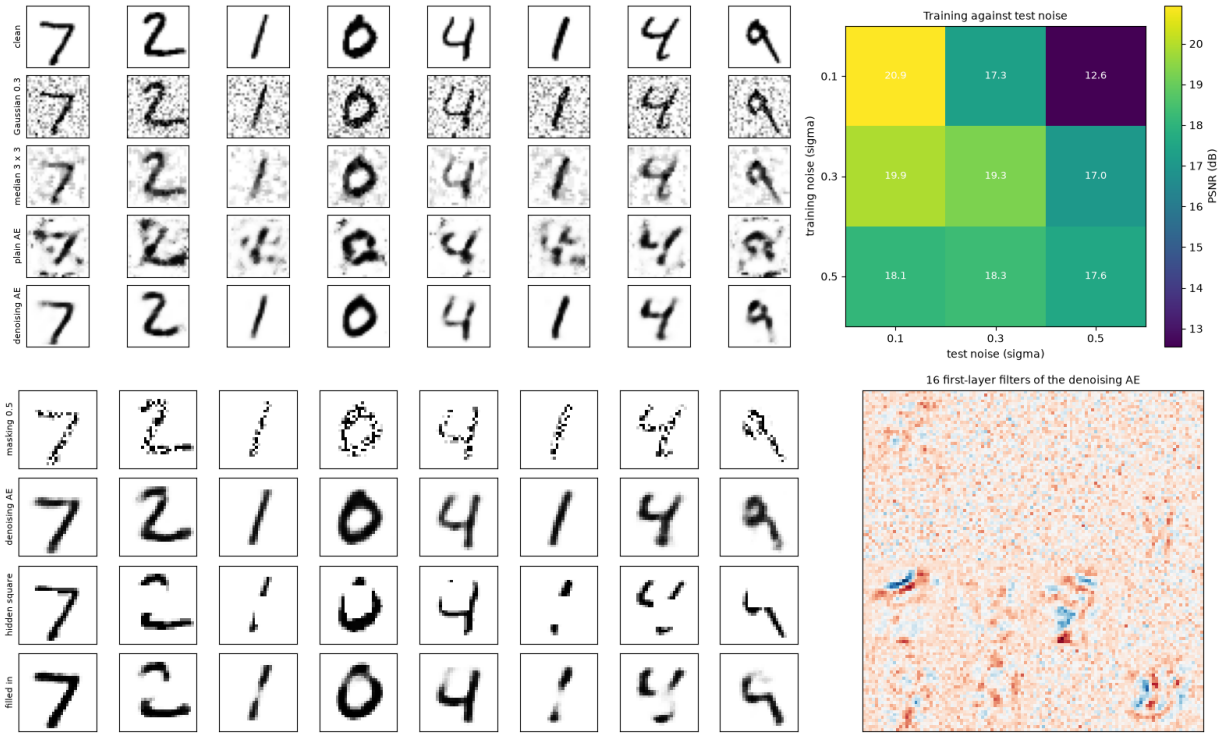
A proofreader trained on clean books, one trained on smudged books, and a simple rule: replace each letter by the most common one around it.

IMPLEMENTATION

```
def median3(X):
    P = np.pad(X.reshape(-1, 28, 28), ((0, 0), (1, 1), (1, 1)))
    stack = np.stack([P[:, i:i + 28, j:j + 28] for i in range(3) for j in range(3)])
    return np.median(stack, 0).reshape(len(X), -1)
def train(Xtr, Xte, noise, seed=0):
    t0 = time.perf_counter(); net, hist = d126.train_ae(Xtr, SIZES, ACTS, "bce", epochs=20,
        seed=seed, Xval=Xte[:2000], noise=noise)
```

ACTUAL OUTPUT

```
three networks trained, 20 epochs each on 20,000 images: 100, 135 and 104 s
  method                                Gaussian 0.3   masking 0.5
  no repair (the damaged image)                13.32         12.45
  3 x 3 median filter                          17.55         10.91
  PCA, 32 components                           16.42         13.63
  plain autoencoder (Day 126)                  12.93         13.05
  denoising, trained on Gaussian 0.3           19.25         13.07
  denoising, trained on masking 0.5             9.45         18.57
Actual output from this lesson's run.
```



Top left: clean, noisy (σ 0.3), median filter, plain autoencoder, denoising autoencoder. Top right: training against test noise. Bottom left: masking and its repair, a hidden square and its fill. Bottom right: 16 first-layer filters of the Gaussian denoiser.

What it means

Each denoiser is the best repair for the damage it was trained on: 19.25 dB on Gaussian noise, 18.57 on masking. The median filter is a strong baseline for noise (17.55) but makes masking worse (10.91): with half the pixels missing, the median of a neighbourhood is often 0. The plain autoencoder never saw noise and makes it worse (12.93 against 13.32 untouched): it turns noise into stray strokes (figure, fourth row).

WHY CHOOSE THIS?

Learns what a digit looks like and restores it.

WHY NOT THIS? (TRADEOFFS)

Each denoiser is tuned to one kind of damage (section 4).

3. Training noise against test noise

DEFINITION — start here, no prior knowledge assumed

Denoisers trained on Gaussian σ = 0.1, 0.3 and 0.5, each tested on all three levels.

REAL-WORLD SCENARIO

A driver who only ever practised in light rain, against one who practised in storms.

ACTUAL OUTPUT

```
  trained on       tested on 0.1   tested on 0.3   tested on 0.5
  sigma 0.1               20.93           17.30           12.55
  sigma 0.3               19.91           19.27           17.02
  sigma 0.5               18.09           18.29           17.57
  no repair               22.65           13.32            9.37
Actual output from this lesson's run.
```

What it means

Each denoiser does best near its own level, and the one trained on 0.3 is the most even (19.91 to 17.02). The surprise is the first column: on light noise (σ = 0.1) the untouched image (22.65 dB) beats every denoiser, even the one trained for it (20.93). An autoencoder always blurs a little, since 32 numbers cannot hold every stroke detail (Day 126's best reconstruction was about 21 dB); when the noise is mild, that blur costs more than the noise.

WHY CHOOSE THIS?

Training on stronger noise gives a more robust denoiser.

WHY NOT THIS? (TRADEOFFS)

On clean or nearly clean images, the denoiser makes them worse.

4. The wrong kind of noise

DEFINITION — start here, no prior knowledge assumed

Each denoiser on the other's damage, and both on salt-and-pepper noise (20% of pixels forced to black or white), which neither saw.

REAL-WORLD SCENARIO

A spell-checker trained on typos, given a document with missing words.

ACTUAL OUTPUT

```
  trained on                  Gaussian 0.3          masking 0.5   salt and pepper 20%
  Gaussian 0.3                       19.25                13.07                18.34
  masking 0.5                         9.45                18.57                10.33
  no repair                          13.32                12.45                10.16
  3 x 3 median                       17.55                10.91                18.49
Actual output from this lesson's run.
```

What it means

The Gaussian denoiser copes with salt and pepper (18.34, about level with the median filter's 18.49), probably because both scatter bright pixels on the background. The masking denoiser does harm on noise it never saw (9.45 on Gaussian, below the untouched 13.32): it learned to fill in missing ink, so it treats every speck of noise as part of a stroke.

WHY CHOOSE THIS?

Measures how far a denoiser's knowledge travels.

WHY NOT THIS? (TRADEOFFS)

Three kinds of damage; real damage is often a mixture.

5. What the code holds when the input is damaged

DEFINITION — start here, no prior knowledge assumed

kNN on the 32-number codes; the reference codes come from clean training images, and only the test images are damaged.

REAL-WORLD SCENARIO

Recognising a friend's voice on a bad phone line.

ACTUAL OUTPUT

```
  network                  clean test images    Gaussian 0.3    Gaussian 0.5
  plain                                0.9155          0.2815          0.1825
  denoising (0.3)                      0.9110          0.9090          0.8625
the reference codes come from clean training images; only the test images are damaged
Actual output from this lesson's run.
```

What it means

On clean images the two codes are about equally good (0.9155 and 0.9110). On noisy images the plain autoencoder's codes collapse (0.2815 at σ 0.3), while the denoiser's hold up (0.9090, and 0.8625 at σ 0.5). Learning to undo damage made its codes describe the digit rather than the pixels.

WHY CHOOSE THIS?

Robust features, learned with no labels.

WHY NOT THIS? (TRADEOFFS)

Robust to the damage it was trained on, not to every kind.

6. Filling in a hidden square

DEFINITION — start here, no prior knowledge assumed

A 10 x 10 square blanked at a random place in each test image; the error is measured only inside the square.

REAL-WORLD SCENARIO

Guessing the words under a coffee stain from the rest of the sentence.

ACTUAL OUTPUT

```
  left blank                       : error inside the square (root mean square) 0.5492; PSNR of
      the whole image 14.15 dB
  PCA, 32 components               : error inside the square (root mean square) 0.4717; PSNR of
      the whole image 13.76 dB
  plain autoencoder                : error inside the square (root mean square) 0.5246; PSNR of
      the whole image 13.55 dB
  denoising, trained on masking 0.5 : error inside the square (root mean square) 0.4354; PSNR of
      the whole image 14.35 dB
masking taught the network to guess missing pixels from the rest of the digit, though never a
    whole square at once
Actual output from this lesson's run.
```

What it means

Every method guesses a little better than leaving the square blank (0.5492), and the masking denoiser does best (0.4354), but the gain is modest: in the figure the fills are faint and some strokes stay broken. Masking hid scattered pixels, never a whole block, so the network rarely had to invent a missing stroke.

WHY CHOOSE THIS?

A start on inpainting with no extra training.

WHY NOT THIS? (TRADEOFFS)

Training on hidden squares would be the honest way to do better; not tried here.

Interview preparation — technical questions

Q: What is a denoising autoencoder?

A: An autoencoder given damaged inputs and trained to output the clean originals.

Q: How do you score image repair?

A: PSNR: 10 $\log_{10}$(1/MSE); here 19.25 dB against 13.32 untouched.

Q: When can a denoiser hurt?

A: On light noise (untouched 22.65 dB beat all) and on damage it never saw (9.45 dB).

Q: Which training noise level?

A: A middle one generalised best here (19.91 to 17.02 dB across levels).

Q: Why are denoising codes useful?

A: They describe the digit, not the pixels: kNN 0.9090 on noisy inputs against 0.2815.

Interview preparation — theoretical questions

Q: Why does damage make the network learn more?

A: It cannot copy; it must learn what clean images look like to undo the damage.

Q: Why does an autoencoder blur?

A: A 32-number code cannot hold every detail; reconstructions lose fine strokes.

Q: Why did the median filter fail on masking?

A: With half the pixels missing, the neighbourhood's median is often 0.

Q: What did the first-layer filters learn?

A: Most look like noise; a few pick out short strokes (figure).

Key takeaway and what's next

KEY TAKEAWAY

A denoising autoencoder learns what images should look like by repairing damage it made itself. It beat the median filter and PCA on the damage it trained on, and its codes stayed meaningful under noise. It is a specialist: mild noise is better left alone, and the wrong kind of damage can make it worse.

- Gaussian 0.3: denoiser 19.25 dB; median 17.55; PCA 16.42; plain autoencoder 12.93; untouched 13.32.
- Masking 0.5: masking denoiser 18.57; median 10.91.
- Light noise (0.1): untouched 22.65 dB beat every denoiser.
- kNN on noisy codes: denoiser 0.9090, plain 0.2815.

Suggested watch order, if starting from zero

- 1. 86 - Applications of Autoencoders - Denoising using MNIST dataset (DigitalSreeni) — a denoising autoencoder on MNIST, in code.
- 2. Lec18 Sparse and Denoising Autoencoders (Deep Learning For Visual Computing - IITKGP) — a lecture on sparse and denoising autoencoders.

Day 128 preview

- **Anomaly detection with autoencoders**: flagging what the network cannot rebuild, with real thresholds.